Module 1 Notes

Module 1: Introduction to Data Structures, Pointers, Arrays, Structures & Sparse Matrices

Data Structures and Applications (1BCS305) — Detailed Exam Notes

Primary source: Horowitz, Sahni & Anderson-Freed, *Fundamentals of Data Structures in C*, 2nd Ed. — Chapter 1 (§1.2), Chapter 2 (§2.1–2.4) **Syllabus mapping:** Introduction & Classification → Pointers & Dynamic Memory → Arrays → Structures/Unions → Polynomials → Sparse Matrices (Representation + Transpose)

> This document is written for revision: each section has **definitions you can quote directly**, **fully worked code**, **step-by-step traces on real data**, and a bank of **likely exam questions** at the end of every section.

Table of Contents

1. Introduction to Data Structures
2. Algorithm Specification
3. Recursive Algorithms
4. Pointers
5. Dynamic Memory Allocation
6. Arrays as an Abstract Data Type
7. Dynamically Allocated Arrays
8. Representation of Multidimensional Arrays
9. Structures and Unions
10. The Polynomial Abstract Data Type
11. The Sparse Matrix Abstract Data Type
12. Full Worked Numeric Example: Sparse Matrix Transpose
13. Consolidated Exam Question Bank

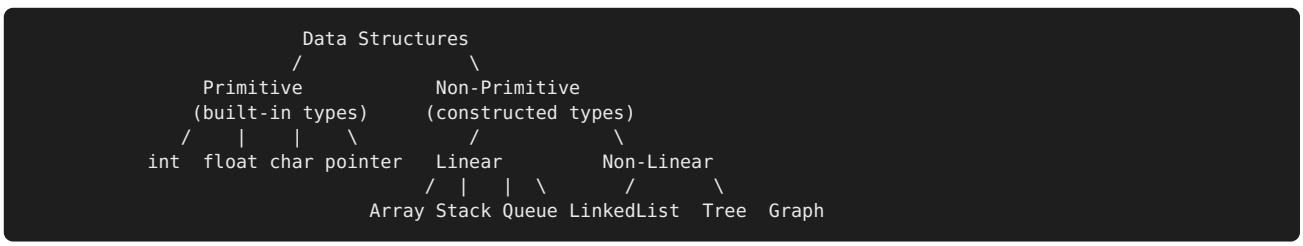
---

1. Introduction to Data Structures

1.1 What is a Data Structure?

A **data structure** is a specific way of organizing, storing, and accessing data in a computer so that operations on it can be performed efficiently. Choosing the right data structure for a problem directly affects the efficiency (time and space) of the algorithms that use it.

1.2 Classification of Data Structures



A. Primitive Data Structures - Directly operated upon by machine-level instructions. - Represent single, basic values. - Examples in C: `int`, `float`, `char`, `double`, pointers. - Size and format are usually fixed and machine-dependent.

B. Non-Primitive Data Structures - Derived from primitive data types; used to represent a *collection* or *group* of data items, possibly with relationships between them. - Further divided by whether elements have a sequential relationship:

Type	Property	Examples

Linear	Elements are arranged in a sequential order; each element (except first/last) has a unique predecessor and successor.	Arrays, Stacks, Queues, Linked Lists
Non-Linear	Elements are not arranged sequentially; an element may connect to multiple other elements.	Trees, Graphs

Why classification matters for exams: A common question is *"Differentiate primitive and non-primitive data structures with examples"* — always mention that primitive types are language-supported basic types, while non-primitive types are user-constructed collections, and give the linear/non-linear split with one example each.

1.3 Data Abstraction and Abstract Data Types (ADT)

An important theme of this entire module is the **Abstract Data Type (ADT)**.

Definition: A data type is a collection of objects and a set of operations that act on those objects. An **Abstract Data Type** is a data type where: 1. The specification of the objects and the operations on them is **separated** from the implementation. 2. Users of the ADT know only *what* operations do (the specification/interface), not *how* they are implemented internally.

Throughout this module, every structure (Array, Polynomial, Sparse Matrix) is first defined as an **ADT** (a `structure ... is / objects: ... / functions: ...` specification block) and *then* an implementation in C is given. **Exams frequently ask you to write the ADT specification of Array / Polynomial / Sparse Matrix — memorize the `structure X is / objects / functions` format shown in each section below.**

---

2. Algorithm Specification

2.1 Definition of an Algorithm

> **An algorithm is a finite set of instructions that, if followed, accomplishes a particular task.**

Every algorithm **must** satisfy all five of the following criteria (a very common short-answer/definition question):

#	Criterion	Meaning
1	**Input**	Zero or more quantities are externally supplied.
2	**Output**	At least one quantity is produced.
3	**Definiteness**	Each instruction must be clear and unambiguous.
4	**Finiteness**	The algorithm must terminate after a finite number of steps, for *all* cases.
5	**Effectiveness**	Every instruction must be basic enough that it could, in principle, be carried out by a person using only pencil and paper.

Algorithm vs. Program: A *program* does not have to satisfy the finiteness property — e.g., an operating system runs in an unending wait loop until it is shut down. In this course (and in practice) *algorithm* and *program* are mostly used interchangeably because our programs are always designed to terminate.

Ways of describing an algorithm: - Natural language (English) — must still be unambiguous. - Flowcharts — good only for small/simple algorithms. - Pseudocode / actual C code — the method used throughout this textbook and course.

2.2 Worked Example 1 — Selection Sort

Problem: Sort `n > 1` integers stored in an array `list`.

Informal English statement (not yet an algorithm — ambiguous): > "From the integers that are currently unsorted, find the smallest and place it next in the sorted list."

Refinement (Program 1.1):

```
        for (i = 0; i < n; i++) {
            /* Examine list[i] to list[n-1] and suppose the smallest is at list[min]; */
```

```
        /* Interchange list[i] and list[min]; */
    }
```

Two subtasks remain: (a) finding the smallest element, (b) swapping two elements. The swap is written using pointers because C is call-by-value — to modify the caller's variables, the function must receive their **addresses**:

```
    void swap(int *x, int *y) {
        int temp = *x;   /* store what x points to, into temp        */
        *x = *y;         /* store what y points to, into x's location */
        *y = temp;        /* store temp into y's location              */
    }
    /* called as: swap(&a, &b); */
```

Full algorithm (Program 1.3 — selection sort core loop):

```
    void sort(int list[], int n) {
        int i, j, min, temp;
        for (i = 0; i < n-1; i++) {
            min = i;
            for (j = i+1; j < n; j++)
                if (list[j] < list[min])
                    min = j;
            SWAP(list[i], list[min], temp);
        }
    }
```

Theorem 1.1 (correctness): Function `sort(list, n)` correctly sorts `n > 1` integers such that `list[0] ≤ list[1] ≤ ... ≤ list[n-1]`. **Proof sketch:** After the outer loop completes iteration `i = q`, `list[q] ≤ list[r]` for all `q < r < n`, and elements `list[0..q]` are never altered again in later iterations. Hence after the final iteration (`i = n-2`), the whole array is sorted. *(This kind of "state an invariant, then argue by induction on the loop" proof is a common exam technique — be ready to reproduce it for other simple algorithms too.)*

2.3 Worked Example 2 — Binary Search (Iterative)

Problem: Given `n` distinct integers already sorted in ascending order in `list`, determine whether `searchnum` is present; return its index if found, else `-1`.

Idea: Maintain `left` and `right` boundaries of the region still to be searched. Compare the middle element to `searchnum`: - If equal → found, return the index. - If `list[middle] < searchnum` → discard the left half, search the right half. - If `list[middle] > searchnum` → discard the right half, search the left half. - Stop (return `-1`) when `left > right`.

This is the basis for the **recursive** version below.

---

3. Recursive Algorithms

3.1 What is Recursion?

A function is: - **Directly recursive** if it calls itself. - **Indirectly recursive** if it calls a second function that (eventually, possibly through further calls) calls the first function again.

Key insight (often tested): *Any* function that can be written using assignment, `if-else`, and `while` statements can also be written recursively — recursion is a general programming technique, not a trick limited to a handful of "special" problems like factorial.

When should you use recursion? When the *problem itself* has a natural recursive definition — e.g.: - Factorial: `n! = n · (n-1)!`, with base case `0! = 1`. - Fibonacci: $f_i = f_{i-1} + f_{i-2}$, base cases $f_0 = 0, f_1 = 1$. - Binomial coefficients: `C(n, m) = C(n-1, m) + C(n-1, m-1)`.

Two requirements for writing a correct recursive function (commonly asked): 1. **Base case(s)** — boundary condition(s) that terminate the recursion without a further recursive call. 2. **Progress toward the base case** — every recursive call must move the problem strictly closer to a base case (e.g., a smaller `n`, or a narrower search range).

3.2 Worked Example — Recursive Binary Search

Converting the iterative binary search into a recursive one: keep the success/failure termination conditions as **base cases**, and replace the loop with a call that narrows `[left, right]`.

```
    int binsearch(int list[], int searchnum, int left, int right) {
```

```
    /* search list[left..right] for searchnum; return index if found, else -1 */
    int middle;
    if (left <= right) {
        middle = (left + right) / 2;
        switch (COMPARE(list[middle], searchnum)) {
            case -1: return binsearch(list, searchnum, middle+1, right); /* search right half */
            case  0: return middle;                                      /* found! */
            case  1: return binsearch(list, searchnum, left, middle-1);  /* search left half */
        }
    }
    return -1;   /* base case: not found, left > right */
}
```

- **Base cases:** `list[middle] == searchnum` (success) and `left > right` (failure).
- **Progress:** each call halves the search interval, so it must eventually hit a base case.
- **Time complexity:** O(log n) — same as the iterative version, since the search space halves each call.

3.3 Worked Example — Generating All Permutations

Problem: Given a set of `n` elements, print all `n!` permutations.

Recursive insight: For a set `{a, b, c, d}`, the permutations are: 1. `a` followed by all permutations of `{b, c, d}` 2. `b` followed by all permutations of `{a, c, d}` 3. `c` followed by all permutations of `{a, b, d}` 4. `d` followed by all permutations of `{a, b, c}`

The phrase **"followed by all permutations"** is the giveaway — solving the `n`-element case reduces to solving `n-1`-element sub-cases.

```
void perm(char *list, int i, int n) {
    /* generate all permutations of list[i..n] */
    int j, temp;
    if (i == n) {                       /* base case: only one element left to permute */
        for (j = 0; j <= n; j++)
            printf("%c", list[j]);
        printf(" ");
    } else {
        /* list[i..n] has more than one permutation: generate recursively */
        for (j = i; j <= n; j++) {
            SWAP(list[i], list[j], temp);
            perm(list, i+1, n);
            SWAP(list[i], list[j], temp);   /* restore for next iteration (backtrack) */
        }
    }
}
/* initial call: perm(list, 0, n-1); */
```

- **Base case:** `i == n` (only one element left — print the current arrangement).
- **Technique used:** this is an early example of **backtracking** — swap to fix an element, recurse, then swap back to restore the original order before trying the next candidate.
- Each recursive call creates its own local copies of `i` (only `i` changes between calls; `n` and the array pointer `list` stay the same).

3.4 Classic Recursion Problems (frequently set as exercises/exam questions)

Problem	Recursive Definition
Factorial	`n! = 1` if `n ≤ 1`; else `n · (n-1)!`
Fibonacci	$f_0=0$, $f_1=1$, $f_i = f_{i-1} + f_{i-2}$ for $i > 1$
Towers of Hanoi	Move `n-1` disks to spare peg → move largest disk to destination → move `n-1` disks from spare to destination
Ackermann's function	`A(m,n) = n+1` if `m=0`; `A(m-1,1)` if `n=0`; else `A(m-1, A(m,n-1))`
Powerset of a set	`powerset({}) = {{}}`; otherwise take powerset of remaining elements, with and without the first element

---

4. Pointers

4.1 Pointer Fundamentals

For every basic type `T` in C, there is a corresponding **pointer-to-T** type. The *value* stored in a pointer variable is a memory **address**.

Two essential operators:

Operator	Name	Effect
`&`	address-of	Returns the memory address of its operand
`*`	dereference / indirection	Accesses the value stored *at* the address held by a pointer

```
int i, *pi;      /* i is an int; pi is a pointer to an int */
pi = &i;          /* pi now holds the address of i */
*pi = 10;         /* stores 10 into the memory location pi points to (i.e., into i) */
i = 10;           /* equivalent direct assignment */
```

Both `i = 10;` and `*pi = 10;` (after `pi = &i;`) store 10 into the *same* memory location.

4.2 Pointer Properties

- Since a pointer is essentially a (non-negative) address, C permits **arithmetic** on pointers: addition, subtraction (and comparison: `<`, `>`, `==`).
- Pointer **size may vary** across systems and even across data types on the same system (e.g., a pointer-to-char and a pointer-to-float need not be the same size on every machine).
- **NULL pointer:** C provides a special value, the null pointer, that points to no object or function — conventionally represented by the integer `0`, and available as the macro `NULL` (defined in `<stddef.h>` for ANSI C, `<stdio.h>` for K&R C).

  ```
  if (pi == NULL)   /* explicit test */
  if (!pi)          /* equivalent shorthand: NULL is treated as false */
  ```

4.3 Why Pointers Can Be Dangerous (common short-answer topic)

1. **Uninitialized/dangling pointers:** always set a pointer to `NULL` when it is not pointing to a valid object — this reduces the chance of accidentally dereferencing an invalid address (out-of-range memory, or memory that doesn't hold what you expect).
   - On some systems dereferencing `NULL` returns `NULL` and execution continues; on others, it triggers whatever bits happen to be at address zero, often producing a serious runtime error.
2. **Implicit int-to-pointer confusion:** many systems give pointers the same size as `int`. Since `int` is the default return type in older C, a function whose return type is accidentally omitted defaults to `int` and can later be misinterpreted as a pointer — always declare explicit return types.
3. **Type mismatches:** always use explicit type casts when converting between pointer types:

   ```
   pi = malloc(sizeof(int));        /* assign to pi, a pointer-to-int */
   pf = (float *) pi;               /* explicit cast: int* -> float* */
   ```

---

5. Dynamic Memory Allocation

5.1 Why Dynamic Allocation?

When writing a program you often don't know in advance how much storage you will need, and you don't want to over-allocate a large fixed-size area that may never be used. C solves this with the **heap** — a pool of memory from which storage can be requested (and returned) **at run time**.

5.2 `malloc` and `free`

Function	Purpose

`malloc(size)`	Requests `size` bytes from the heap. Returns a pointer to the start of the allocated block (type `void *` in ANSI C, so an explicit cast to the required pointer type is standard practice), or fails if the memory is unavailable.
`free(ptr)`	Releases a previously `malloc` -allocated block back to the system, making it available for future allocation.

```
        int   i, *pi;
        float f, *pf;

        pi = (int *)   malloc(sizeof(int));    /* request space for one int   */
        pf = (float *) malloc(sizeof(float));  /* request space for one float */

        *pi = 1024;
        *pf = 3.14;
        printf("an integer = %d, a float = %f\n", *pi, *pf);

        free(pi);
        free(pf);
```

- `sizeof(int)` / `sizeof(float)` tells `malloc` exactly how many bytes are required — this is the standard idiom and should always be used instead of hard-coded numbers (which are not portable across machines).
- After `free(ptr)`, using `ptr` again is **undefined behaviour / improper** — the memory may be reused by a later allocation.

5.3 Dangling References (important exam definition)

> A **dangling reference** occurs when all pointers to a dynamically allocated block of memory are lost (e.g., overwritten) *before* the block has been freed — the memory can no longer be accessed or released, and is effectively "lost" to the program for its remaining lifetime (a **memory leak**).

Example:

```
        pf = (float *) malloc(sizeof(float));
        *pf = 3.14;
        ...
        pf = (float *) malloc(sizeof(float));  /* BUG: old block's address is overwritten
                                                  before it was freed — old block is now
                                                  unreachable = dangling reference/leak  */
```

Rule of thumb taught in the text: always `free()` dynamically allocated storage as soon as it is no longer needed, and never overwrite the only pointer to a block before freeing it.

5.4 Relation to Self-Referential Structures

Structures with pointer members that reference the *same* structure type (see §9.4) are the main reason dynamic memory management matters in this course — building linked structures (covered in the next module) requires creating (`malloc`) and destroying (`free`) individual nodes at run time, since the number of nodes is not known in advance.

---

6. Arrays as an Abstract Data Type

6.1 The ADT View of an Array

Most programmers think of an array only as "a block of consecutive memory locations" — this is an *implementation* detail. Abstractly, an array is a set of `<index, value>` pairs: a mapping (correspondence) from a set of valid indices to a set of values.

ADT specification (memorize this format — commonly asked to reproduce):

```
structure Array is
  objects: A set of pairs <index, value> where for each value of index there is a
  value from the set item. Index is a finite ordered set of one or more dimensions.
  functions:
     for all A ∈ Array, i ∈ index, x ∈ item, j, size ∈ integer

     Array   Create(j, list) ::= return an array of j dimensions where list is a
                                 j-tuple whose i-th element is the size of the
```

```
                                i-th dimension. Items are undefined.
     Item    Retrieve(A, i)  ::= if (i ∈ index) return the item associated with
                                index value i in array A, else return error
     Array   Store(A, i, x)  ::= if (i ∈ index) return an array identical to A
                                except the new pair <i, x> has been inserted,
                                else return error
end Array
```

This definition highlights that an array supports essentially two operations: **retrieve** a value at an index, and **store** a value at an index — everything else (searching, sorting, etc.) is built from these.

6.2 Arrays in C — Declaration and Addressing

```
int list[5], *plist[5];
```

- `list` — an array of 5 integers.
- `plist` — an array of 5 *pointers to integers* (note the difference — `*` binds to each element, not to `list` as a whole).
- **C arrays are always 0-indexed**: valid subscripts for `list` are `0..4`.

Memory layout: the compiler allocates 5 *consecutive* memory locations. The address of `list[0]` is called the **base address**, say `a`. Then:

Element	Address
`list[0]`	`a`
`list[1]`	`a + sizeof(int)`
`list[2]`	`a + 2·sizeof(int)`
`list[3]`	`a + 3·sizeof(int)`
`list[i]`	`a + i·sizeof(int)`

Writing `list[i]` in C is **exactly equivalent** to `*(list + i)` — C automatically scales the offset `i` by `sizeof(int)` so you never multiply by the element size yourself.

Pointer vs array declarations — a classic "spot the difference" question:

```
int *list1;     /* list1 is just a pointer — NO storage for integers is reserved   */
int list2[5];   /* list2 is an array — 5 integer-sized memory locations ARE reserved */
```

- `list2` behaves as a pointer to `list2[0]`; `list2 + i` is a pointer to `list2[i]`.
- Regardless of type, `(list2 + i)` always equals `&list2[i]`, so `*(list2 + i)` always equals `list2[i]`.

6.3 Arrays as Function Parameters

A one-dimensional array's *size* is defined only where it is declared (typically in `main`); a function receiving the array does **not** automatically know its size — new storage is *not* allocated inside the function. The size must therefore be passed explicitly as an extra parameter (or made available as a global).

```
float sum(float list[], int n) {
    int i;
    float tempsum = 0;
    for (i = 0; i < n; i++)
        tempsum += list[i];
    return tempsum;
}
/* called as: answer = sum(input, MAX_SIZE); */
```

Important subtlety (frequently tested as a "true/false" or short-answer question): Although C uses **call-by-value** parameter passing, when `input` (an array) is passed to `sum`, the value copied into the parameter `list` is really `&input[0]` — a pointer. So while the *pointer itself* is copied by value, it still points at the caller's original data. This means: > **Changes made to array elements *inside* the function *do* persist back in the caller**, even though C's parameter-passing mechanism is technically call-by-value. (This is different from passing a plain `int` or `float`, whose modifications inside a function do *not* propagate back.)

6.4 Pointer-based Array Traversal — Worked Example

```
void print1(int *ptr, int rows) {
```

```
    /* print a one-dimensional array using pointer arithmetic */
    int i;
    printf("Address Contents\n");
    for (i = 0; i < rows; i++)
        printf("%8u %5d\n", ptr + i, *(ptr + i));
}
/* called as: print1(&one[0], 5); */
```

For `int one[] = {0, 1, 2, 3, 4};`, this prints each element's address (incrementing by `sizeof(int)` each time, e.g. by 2 bytes on some older systems / 4 bytes typically on modern ones) alongside its value — illustrating exactly how `ptr + i` and `*(ptr + i)` relate to `one[i]`.

---

7. Dynamically Allocated Arrays

7.1 Motivation

A fixed-size array declaration such as `int list[100];` forces you to guess a size in advance — too small and you overflow; too large and you waste memory. **Dynamically allocated arrays** solve this by deferring the size decision to run time, using `malloc`.

```
int *arr;
int n;

printf("Enter the number of elements: ");
scanf("%d", &n);

arr = (int *) malloc(n * sizeof(int));   /* allocate exactly n integers, at run time */
if (arr == NULL) {
    printf("Memory allocation failed\n");
    exit(1);
}

for (int i = 0; i < n; i++)
    scanf("%d", &arr[i]);      /* arr can be indexed exactly like a normal array */

/* ... use arr ...*/

free(arr);                     /* release the memory once done */
```

7.2 Key Points (commonly tested)

- `arr` is declared as a plain pointer (`int *arr;`), not as `int arr[SIZE];` — the array itself does not exist until `malloc` is called.
- `arr[i]` and `*(arr + i)` are usable interchangeably, exactly as with a statically declared array, because a dynamically allocated block is also just a contiguous region of memory with `arr` pointing to its first byte.
- The size expression passed to `malloc` (`n * sizeof(int)`) must be recomputed if the type or count changes — always use `sizeof(type)`, never a hard-coded byte count.
- Always **check the return value of `malloc`** for `NULL` before use (allocation can fail if memory is exhausted).
- Always `free` the array once it is no longer needed, to avoid memory leaks.
- Dynamically allocated arrays are the foundation for representing **sparse matrices** and **polynomials** efficiently later in this module — the array of `term` triples (§11) and the shared `terms[]` array for polynomials (§10) are both, in the general case, sized based on run-time input rather than a fixed worst case.

---

8. Representation of Multidimensional Arrays

8.1 Element Count

For an array declared $a[upper_0][upper_1]\ldots[upper_{n-1}]$, the total number of elements is the product of all the $upper_i$ values: **total elements = $upper_0 \times upper_1 \times \ldots \times upper_{n-1}$** Example: `a[10][10][10]` requires `10 × 10 × 10 = 1000` storage units.

8.2 Row-Major vs Column-Major Order

There are two standard ways to lay out a multidimensional array in linear (1-D) memory:

Layout	Storage order	Used by

Row-major order	All elements of row 0, then all of row 1, then row 2, …	C, C++, Python (NumPy default)
Column-major order	All elements of column 0, then column 1, …	Fortran, MATLAB

Why this matters: knowing the layout lets you derive a direct **addressing formula** to compute the memory location of `a[i][j]` without scanning the array — this gives O(1) random access to any element of a multidimensional array, exactly as with a 1-D array.

General idea for row-major addressing of a 2-D array `a[M][N]` with base address `base` and element size `w`: **address(a[i][j]) = base + (i × N + j) × w** (i.e., skip `i` complete rows of `N` elements each, then move `j` more elements into the target row.)

---

9. Structures and Unions

9.1 Why Structures?

Arrays group data of the **same** type. C's `struct` mechanism groups data items that may be of **different** types — each field (member) is identified by its own type and name. A structure is equivalent to what other languages call a **record**.

```
struct {
    char  name[10];
    int   age;
    float salary;
} person;

strcpy(person.name, "james");   /* '.' is the structure member (dot) operator */
person.age    = 10;
person.salary = 35000;
```

9.2 Named Structure Types with `typedef`

```
typedef struct human_being {
    char  name[10];
    int   age;
    float salary;
} human_being;

human_being person1, person2;
```

- `human_being` becomes a new type name that can be used just like `int` or `float` for future declarations.

Structure assignment vs comparison (a favourite exam trap): - **ANSI C permits whole-structure assignment**: `person1 = person2;` copies every field at once. - **ANSI C does *NOT* permit direct structure comparison**: `person1 == person2` is **illegal** — you must write a function that compares field by field.

```
#define TRUE  1
#define FALSE 0

int humans_equal(human_being person1, human_being person2) {
    /* return TRUE if person1 and person2 represent the same human, else FALSE */
    if (strcmp(person1.name, person2.name)) return FALSE;
    if (person1.age    != person2.age)      return FALSE;
    if (person1.salary != person2.salary)   return FALSE;
    return TRUE;
}
```

Nested structures: a structure can itself contain another structure as a field:

```
typedef struct { int month; int day; int year; } date;

typedef struct human_being {
    char  name[10];
    int   age;
    float salary;
    date  dob;          /* nested structure */
} human_being;

person1.dob.month = 2;
person1.dob.day   = 11;
person1.dob.year  = 1944;
```

9.3 Unions

A `union` looks syntactically like a `struct`, but **all its members share the same memory location** — only one member is "active"/meaningful at any given time. This is used to save memory when a variable may, at different times, need to hold one of several different kinds of data.

```
typedef struct sex_type {
    enum tag_field { female, male } sex;   /* the "tag" field */
    union {
        int children;   /* meaningful only if sex == female */
        int beard;       /* meaningful only if sex == male   */
    } u;
} sex_type;

person1.sex_info.sex        = male;
person1.sex_info.u.beard    = FALSE;

person2.sex_info.sex          = female;
person2.sex_info.u.children   = 4;
```

Critical exam point: C does **not** check that you are using the "correct" union member for the current tag value — it is entirely the programmer's responsibility to first set a tag field and then be disciplined about only reading/writing the matching union member. Using the wrong member gives meaningless results without any compiler warning.

Struct vs Union — summary table (very common comparison question):

Aspect	`struct`	`union`
Memory	Each member gets its **own** space; total size ≈ sum of member sizes (+ padding)	All members **share** the same space; total size = size of the **largest** member (+ padding)
Active members	All members are valid simultaneously	Only **one** member is valid/meaningful at a time
Typical use	Grouping *related but different* pieces of data that all exist together (e.g., name+age+salary)	Representing *one of several possible* data types at a given time (space-efficient "variant" data)

9.4 Internal Implementation of Structures

- Compilers generally store structure fields in **increasing memory addresses**, in the order they are declared.
- **Padding/holes** may be inserted between fields so that each field starts at a properly *aligned* memory address (e.g., a multiple of 4, 8, or 16 bytes, depending on the type and the machine).
- The overall size of a `struct` / `union` object equals the storage needed for the largest required alignment plus all members (and any padding) — structures must begin and end on a proper memory boundary.

9.5 Self-Referential Structures

> **Definition:** A self-referential structure is one in which one or more of its own components is a **pointer to another instance of the same structure type**.

```
typedef struct list {
    char data;
    struct list *link;     /* pointer to another 'list' node, or NULL */
} list;
```

- `data` holds a single character; `link` holds either the address of another `list` node, or the null pointer.
- Self-referential structures inherently require **dynamic storage management** (`malloc` / `free`) since we typically don't know in advance how many nodes (instances) we will need.

Worked example — linking three nodes manually:

```
list item1, item2, item3;
item1.data = 'a';
item2.data = 'b';
item3.data = 'c';
item1.link = item2.link = item3.link = NULL;   /* all initially isolated */
```

```
        item2.link = &item3;    /* link item2 -> item3 */
        item1.link = &item2;    /* link item1 -> item2 */
```

Diagrammatically, after these two assignments:

```
item1 ('a') --link--> item2 ('b') --link--> item3 ('c') --link--> NULL
```

This is precisely the conceptual building block of a **linked list** — the topic of the next module. Understanding self-referential structures thoroughly here makes linked lists much easier later.

---

10. The Polynomial Abstract Data Type

10.1 Motivation

Arrays are not just data structures in their own right — they can be used to **implement** other, higher-level ADTs. The polynomial is the textbook's running example of this idea.

A polynomial is a sum of terms, each of the form `a·x^e` where `x` is the variable, `a` is the coefficient, and `e ≥ 0` is the exponent. The **degree** of a polynomial is its largest (leading) exponent.

Example polynomials: **A(x) = 2x¹⁰⁰⁰ + 1** and **B(x) = x⁴ + 10x³ + 3x² + 1**

Standard mathematical operations: given `A(x) = Σ aᵢxⁱ` and `B(x) = Σ bᵢxⁱ`: **A(x) + B(x) = Σ (aᵢ + bᵢ) xⁱ**

10.2 Polynomial ADT Specification (memorize this format)

```
structure Polynomial is
  objects: p(x) = a1 x^e1 + ... + an x^en; a set of ordered pairs of <e, a> where
           coefficients are in Coefficients and exponents e are integers >= 0
  functions:
     for all poly, poly1, poly2 ∈ Polynomial, coef ∈ Coefficients, expon ∈ Exponents

     Polynomial  Zero()                       ::= return the polynomial p(x)=0
     Boolean     IsZero(poly)                 ::= if (poly) return FALSE, else return TRUE
     Coefficient Coef(poly, expon)             ::= if (expon ∈ poly) return its coefficient,
                                                       else return zero
     Exponent    Lead-Exp(poly)                 ::= return the largest exponent in poly
     Polynomial  Attach(poly, coef, expon)      ::= if (expon ∈ poly) return error,
                                                       else return poly with <coef, expon> inserted
     Polynomial  Remove(poly, expon)            ::= if (expon ∈ poly) return poly with the term
                                                       for expon deleted, else return error
     Polynomial  SingleMult(poly, coef, expon)   ::= return poly · coef · x^expon
     Polynomial  Add(poly1, poly2)               ::= return poly1 + poly2
     Polynomial  Mult(poly1, poly2)               ::= return poly1 · poly2
end Polynomial
```

10.3 Representation 1 — One Array Per Polynomial (simple but wasteful)

```
        #define MAX_DEGREE 101
        typedef struct {
            int   degree;
            float coef[MAX_DEGREE];      /* coef[i] = coefficient of x^(n-i) */
        } polynomial;
```

`a.coef[i]` holds the coefficient of `x^(n-i)` if that term exists, `0` otherwise — coefficients are stored in *decreasing* order of exponent.

Problem: this wastes a large amount of space whenever `a.degree ≪ MAX_DEGREE`, or whenever the polynomial is **sparse** (most coefficients are zero) — exactly the same problem we will see with sparse matrices.

Initial (representation-independent) version of `Add`, in terms of the ADT operations:

```
        /* d = a + b, where a, b, and d are polynomials */
        d = Zero();
        while (!IsZero(a) && !IsZero(b)) {
            switch (COMPARE(Lead-Exp(a), Lead-Exp(b))) {
                case -1:                                    /* a's leading exp < b's */
                    d = Attach(d, Coef(b, Lead-Exp(b)), Lead-Exp(b));
```

```
                b = Remove(b, Lead-Exp(b));
                break;
            case 0:                                     /* equal exponents */
                sum = Coef(a, Lead-Exp(a)) + Coef(b, Lead-Exp(b));
                if (sum) {
                    Attach(d, sum, Lead-Exp(a));
                    a = Remove(a, Lead-Exp(a));
                    b = Remove(b, Lead-Exp(b));
                }
                break;
            case 1:                                     /* a's leading exp > b's */
                d = Attach(d, Coef(a, Lead-Exp(a)), Lead-Exp(a));
                a = Remove(a, Lead-Exp(a));
        }
    }
    /* insert any remaining terms of a or b into d */
```

10.4 Representation 2 — Single Shared Array (space-efficient; the exam-favourite representation)

To avoid wasting space, use **one global array**, `terms`, shared by *all* polynomials in the program. Each polynomial is represented not by its own array, but by a `<start, finish>` index range into this shared array.

```
#define MAX_TERMS 100
typedef struct {
    float coef;
    int   expon;
} polynomial;

polynomial terms[MAX_TERMS];
int avail = 0;        /* index of the next free location in terms[] */
```

Worked example — storing `A(x) = 2x^1000 + 1` and `B(x) = x^4 + 10x^3 + 3x^2 + 1`:

index	0	1	2	3	4	5
coef	2	1	1	10	3	1
expon	1000	0	4	3	2	0

Here `starta = 0, finisha = 1` (A occupies indices 0–1), `startb = 2, finishb = 5` (B occupies indices 2–5), and `avail = 6` (next free slot).

- If a polynomial `A` has `n` nonzero terms, then `finisha = starta + n - 1`.
- **Space comparison:** for `A(x) = 2x^1000+1`, representation 1 would need 1001 storage units (`coef[0..1000]`), but representation 2 needs only 6 units total for *both* A and B combined (2 index variables + 2 coefficients + 2 exponents for A, similarly small for B). However, if a polynomial has **most terms nonzero** (dense), representation 2 needs roughly *twice* the space of representation 1 (since it stores both a coefficient **and** an exponent for every term, whereas representation 1 stores only coefficients, with the exponent implied by array position). **Rule of thumb:** representation 2 wins whenever polynomials are sparse; representation 1 can win when they are dense.

Full `padd` function using the shared array (this exact code is a strong candidate for a "write the program" exam question):

```
void attach(float coefficient, int exponent) {
    /* add a new term to the polynomial being built */
    if (avail >= MAX_TERMS) {
        fprintf(stderr, "Too many terms in the polynomial\n");
        exit(1);
    }
    terms[avail].coef  = coefficient;
    terms[avail++].expon = exponent;
}

void padd(int starta, int finisha, int startb, int finishb,
          int *startd, int *finishd) {
    /* add A(x) and B(x) to obtain D(x) */
    float coefficient;
    *startd = avail;
    while (starta <= finisha && startb <= finishb) {
        switch (COMPARE(terms[starta].expon, terms[startb].expon)) {
            case -1:                                    /* a's expon < b's expon */
                attach(terms[startb].coef, terms[startb].expon);
                startb++;
                break;
            case 0:                                     /* equal exponents */
```

```
                    coefficient = terms[starta].coef + terms[startb].coef;
                    if (coefficient)
                        attach(coefficient, terms[starta].expon);
                    starta++;
                    startb++;
                    break;
                case 1:                                     /* a's expon > b's expon */
                    attach(terms[starta].coef, terms[starta].expon);
                    starta++;
            }
        }
        /* copy any remaining terms of A(x) */
        for ( ; starta <= finisha; starta++)
            attach(terms[starta].coef, terms[starta].expon);
        /* copy any remaining terms of B(x) */
        for ( ; startb <= finishb; startb++)
            attach(terms[startb].coef, terms[startb].expon);

        *finishd = avail - 1;
    }
```

How it works, step by step: 1. Scan both polynomials from their highest exponent (`starta` , `startb`) toward the lowest, comparing exponents term by term. 2. If exponents differ, copy the term with the **larger** exponent into `terms[]` (via `attach`) and advance only that polynomial's pointer. 3. If exponents are **equal**, add the coefficients; if the sum is nonzero, attach the combined term; advance **both** pointers either way (a zero-sum term is simply dropped). 4. Once one polynomial is exhausted, copy all remaining terms of the other polynomial directly (they have no matching term to combine with).

Time complexity analysis of `padd` : let `m` and `n` be the number of nonzero terms in `A` and `B` respectively. - The `while` loop executes O(1) work per iteration, and each iteration advances `starta` or `startb` (or both) by 1 — so the loop runs at most `m + n − 1` times in the worst case (e.g., when no two exponents ever coincide, such as `A(x) = Σx^{2i}` and `B(x) = Σx^{2i+1}`). - The two clean-up `for` loops together do at most `O(m + n)` additional work. - **Overall time complexity: O(m + n).**

10.5 Comparing the Two Representations (likely comparison question)

	Representation 1 (one array per polynomial)	Representation 2 (shared array with start/finish)
Storage per term	Only coefficient (exponent implied by index)	Coefficient **and** exponent explicitly stored
Space for sparse polynomials	Wasteful — reserves space for every possible exponent up to `MAX_DEGREE`	Efficient — only nonzero terms occupy space
Space for dense polynomials	Efficient	Roughly double, since exponents are stored explicitly too
Number of polynomials supported	One array must be declared per polynomial	Many polynomials can share one global array, limited only by total nonzero terms ≤ `MAX_TERMS`

---

11. The Sparse Matrix Abstract Data Type

11.1 The Problem with the Standard Representation

A matrix with `m` rows and `n` columns ("m × n") is normally represented in C as a 2-D array `a[MAX_ROWS][MAX_COLS]` , giving direct O(1) access to any element `a[i][j]` .

> **Definition:** A matrix is called **sparse** when a large fraction of its elements are zero. (There is no single precise numeric threshold — the textbook's example has only 8 nonzero entries out of 36, i.e. roughly 22% nonzero, and calls that "certainly sparse.")

Why the standard 2-D array representation is a poor fit for sparse matrices: 1. It stores *every* entry — including all the zeros — wasting large amounts of memory when the matrix is large and mostly zero. 2. Compilers typically impose limits on array sizes, so a very large sparse matrix (e.g., 1000 × 1000) may not even be representable as a full 2-D array in the first place.

Conclusion: we should design a representation that stores **only the nonzero elements**.

11.2 Sparse Matrix ADT Specification

```
structure Sparse-Matrix is
  objects: a set of triples, <row, column, value>, where row and column are
           integers forming a unique combination, and value comes from item.
  functions:
     for all a, b ∈ Sparse-Matrix, x ∈ item, i, j, max-col, max-row ∈ index

     Sparse-Matrix Create(max-row, max-col) ::= return a Sparse-Matrix that can hold
                         up to max-items = max-row × max-col, with the given
                         maximum row and column sizes.
     Sparse-Matrix Transpose(a)             ::= return the matrix produced by
                         interchanging the row and column value of every triple.
     Sparse-Matrix Add(a, b)                ::= if dimensions of a and b match,
                         return the matrix from adding corresponding items
                         (same row & column); else return error.
     Sparse-Matrix Multiply(a, b)           ::= if columns of a = rows of b,
                         return the product matrix d, where
                         d[i][j] = Σₖ a[i][k]·b[k][j]; else return error.
end Sparse-Matrix
```

*(Module 1 focuses on the **Representation** and **Transpose** operation specifically — Addition/Multiplication are part of the ADT specification but the transpose algorithm below is the one you need to trace and code for exams.)*

11.3 Representation — Array of Triples

Every nonzero element is uniquely identified by the triple `<row, col, value>`. We store these triples in an array, ordered so that: - Row indices are in **ascending order** (this is essential for the transpose algorithm to work efficiently — see below). - Within each row, column indices are also in **ascending order**.

We also need to know the total number of rows, columns, and nonzero terms — the textbook's convention packs all of this into the *same* array by using index `0` as a header record:

```c
#define MAX_TERMS 101
typedef struct {
    int row;
    int col;
    int value;
} term;

term a[MAX_TERMS];
```

Convention:

Slot	Meaning
`a[0].row`	total number of rows in the matrix
`a[0].col`	total number of columns in the matrix
`a[0].value`	total number of nonzero terms
`a[1] … a[n]`	the actual nonzero triples, in row-major order (ascending row, then ascending column within a row)

11.4 Worked Example — Representing a Matrix

Consider the following 6×6 sparse matrix (from the textbook, Figure 2.3b):

```
       col0 col1 col2 col3 col4 col5
row0    15    0    0   22    0  -15
row1     0   11    3    0    0    0
row2     0    0    0   -6    0    0
row3     0    0    0    0    0    0
row4    91    0    0    0    0    0
row5     0    0   28    0    0    0
```

Only 8 of the 36 entries are nonzero. Stored as an array of triples (row-major, ascending column within row):

index	row	col	value
`a[0]`	6	6	8

`a[1]`	0	0	15
`a[2]`	0	3	22
`a[3]`	0	5	-15
`a[4]`	1	1	11
`a[5]`	1	2	3
`a[6]`	2	3	-6
`a[7]`	4	0	91
`a[8]`	5	2	28

Space saved: the full 2-D array needs 36 storage units; the triple representation needs only 9 triples × 3 fields = 27 units here — and the saving grows dramatically for larger, sparser matrices (e.g., a 1000×1000 matrix with only 2000 nonzero entries needs roughly 6000 units instead of 1,000,000).

11.5 Transposing a Sparse Matrix

Definition: the **transpose** of a matrix interchanges its rows and columns — every element `a[i][j]` of the original becomes element `b[j][i]` of the transpose.

Why the "obvious" approach fails: if we process the original matrix's triples in their given (row-major) order and simply swap each triple's row/col, the *resulting* triples are **not** automatically in row-major order for the transpose:

```
(0,0,15) → becomes (0,0,15)
(0,3,22) → becomes (3,0,22)
(0,5,-15) → becomes (5,0,-15)
```

Placing these consecutively in `b` would put `(3,0,22)` before `(5,0,-15)`, but if a later-processed original triple belonged to, say, row 1 (which becomes column-1-derived row… i.e., an earlier destination row than 3), it would need to be inserted *before* already-placed elements — requiring expensive data movement to keep `b` in correct row-major order.

Correct strategy — process by column of the original matrix: > "Find all the elements in column 0 of `a` and store them as row 0 of `b`; find all the elements in column 1 of `a` and store them as row 1 of `b`; and so on."

Because the original matrix's rows are already stored in ascending row order, scanning column-by-column naturally produces the transpose's rows in the correct ascending order too — **no data movement/reordering is required**.

11.5.1 Algorithm `transpose` (simple version)

```
void transpose(term a[], term b[]) {
    /* b is set to the transpose of a */
    int n, i, j, currentb;
    n = a[0].value;                 /* total number of nonzero elements */
    b[0].row   = a[0].col;          /* rows in b = columns in a */
    b[0].col   = a[0].row;          /* columns in b = rows in a */
    b[0].value = n;

    if (n > 0) {                     /* nonzero matrix */
        currentb = 1;
        for (i = 0; i < a[0].col; i++)           /* for every column i of a ... */
            for (j = 1; j <= n; j++)              /* ...scan all n triples for one from column i */
                if (a[j].col == i) {
                    b[currentb].row   = a[j].col;
                    b[currentb].col   = a[j].row;
                    b[currentb].value = a[j].value;
                    currentb++;
                }
    }
}
```

Time complexity analysis: - The outer loop runs `a[0].col` times (once per column). - For **each** column, the inner loop scans **all** `n = a[0].value` triples to find the ones belonging to that column. - Total time: **O(columns × elements)**. - This is disappointing: transposing the *equivalent* full 2-D array (`for j ... for i ... b[j][i] = a[i][j];`) takes only **O(rows × columns)** time. When the number of nonzero elements is on the order of `rows × columns` (i.e., the matrix isn't actually very sparse), `O(columns × elements)` becomes `O(columns² × rows)` — *worse* than the simple 2-D array transpose! We have traded away too much time for the space we saved.

11.5.2 Algorithm `fast_transpose` (efficient version — the important one for exams)

Idea: avoid rescanning all `n` triples for every single column by doing a **single pass** with the help of two small auxiliary arrays: 1. `row_terms[i]` — the number of nonzero terms in column `i` of `a` (which will become the number of terms in row `i` of `b`). 2. `starting_pos[i]` — the index in `b` where row `i` of the transpose should begin.

```
#define MAX_COL 50
void fast_transpose(term a[], term b[]) {
    /* the transpose of a is placed in b, in O(columns + elements) time */
    int row_terms[MAX_COL], starting_pos[MAX_COL];
    int i, j, num_cols = a[0].col, num_terms = a[0].value;

    b[0].row   = num_cols;
    b[0].col   = a[0].row;
    b[0].value = num_terms;

    if (num_terms > 0) {
        /* Step 1: count how many nonzero terms are in each column of a */
        for (i = 0; i < num_cols; i++)
            row_terms[i] = 0;
        for (i = 1; i <= num_terms; i++)
            row_terms[a[i].col]++;

        /* Step 2: compute the starting index of each row of b */
        starting_pos[0] = 1;
        for (i = 1; i < num_cols; i++)
            starting_pos[i] = starting_pos[i-1] + row_terms[i-1];

        /* Step 3: place every triple of a directly into its slot in b */
        for (i = 1; i <= num_terms; i++) {
            j = starting_pos[a[i].col]++;
            b[j].row   = a[i].col;
            b[j].col   = a[i].row;
            b[j].value = a[i].value;
        }
    }
}
```

How each step works: - **Step 1** tallies, for every column of `a`, how many nonzero entries it has — this becomes the number of entries that row will have in `b`. - **Step 2** turns these per-row counts into a **running/cumulative starting index**: row `i`'s data in `b` begins right after row `i-1`'s data ends (`starting_pos[i] = starting_pos[i-1] + row_terms[i-1]`). - **Step 3** makes a **single pass** over `a`'s triples in their existing (row-major) order. Each triple is dropped directly into its correct final position in `b`, using `starting_pos[]` as a "next free slot" counter for each destination row (`starting_pos[a[i].col]++` both reads the current free slot **and** advances it for the next term of the same column).

Time complexity analysis: - The four loops (zeroing `row_terms`, counting, computing `starting_pos`, placing terms) run `num_cols`, `num_terms`, `num_cols − 1`, and `num_terms` times respectively — each with O(1) work per iteration. - **Total time: O(columns + elements)** — a substantial improvement over `transpose`'s `O(columns × elements)`. - When the number of elements is on the order of `rows × columns` (dense case), this becomes `O(columns × rows)` — matching the plain 2-D array transpose's time, though `fast_transpose` has a larger constant factor. - **Space trade-off:** `fast_transpose` needs two extra arrays (`row_terms`, `starting_pos`), each of size `MAX_COL`, so it uses more space than `transpose`. (This extra space can be reduced to a single array by reusing `row_terms`'s storage for `starting_pos` as each starting position is computed — a common follow-up exam question.)

11.6 Comparison Table (very likely exam question)

Method	Time Complexity	Space	Comment
Full 2-D array transpose	O(rows × columns)	O(rows × columns)	Simple; wastes space when matrix is sparse
`transpose` (array of triples)	O(columns × elements)	O(elements)	Saves space; can be *slower in time* than the 2-D approach if the matrix isn't very sparse
`fast_transpose` (array of triples)	O(columns + elements)	O(elements) + O(columns) extra	Saves **both** time and space for genuinely sparse matrices; needs 2 auxiliary arrays

---

12. Full Worked Numeric Example: Sparse Matrix Transpose

Let's trace `fast_transpose` **by hand** on the matrix from §11.4, to build the exact skill needed for a "trace the algorithm" exam question.

Input `a` (the original matrix, 6 rows, 6 cols, 8 nonzero terms):

index	row	col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

Step 1 — count `row_terms[i]` (nonzero terms in each *column* of `a`, for `i = 0..5`): - Column 0: `a[1]` (row0,col0), `a[7]` (row4,col0) → **2** - Column 1: `a[4]` (row1,col1) → **1** - Column 2: `a[5]`, `a[8]` → **2** - Column 3: `a[2]`, `a[6]` → **2** - Column 4: none → **0** - Column 5: `a[3]` → **1**

`row_terms = [2, 1, 2, 2, 0, 1]`

Step 2 — compute `starting_pos[i]` (cumulative starting index for each row of `b`, `starting_pos[0] = 1`): - `starting_pos[0] = 1` - `starting_pos[1] = starting_pos[0] + row_terms[0] = 1 + 2 = 3` - `starting_pos[2] = starting_pos[1] + row_terms[1] = 3 + 1 = 4` - `starting_pos[3] = starting_pos[2] + row_terms[2] = 4 + 2 = 6` - `starting_pos[4] = starting_pos[3] + row_terms[3] = 6 + 2 = 8` - `starting_pos[5] = starting_pos[4] + row_terms[4] = 8 + 0 = 8`

`starting_pos = [1, 3, 4, 6, 8, 8]`

Step 3 — place every triple of `a`, in order, using `starting_pos[a[i].col]++`:

Process a[i]	a[i].col	j = starting_pos[col] (then ++)	Result written to b[j]
a[1]=(0,0,15)	0	j=1 (→2)	b[1]=(row=0,col=0,val=15)
a[2]=(0,3,22)	3	j=6 (→7)	b[6]=(row=3,col=0,val=22)
a[3]=(0,5,-15)	5	j=8 (→9)	b[8]=(row=5,col=0,val=-15)
a[4]=(1,1,11)	1	j=3 (→4)	b[3]=(row=1,col=1,val=11)
a[5]=(1,2,3)	2	j=4 (→5)	b[4]=(row=2,col=1,val=3)
a[6]=(2,3,-6)	3	j=7 (→8)	b[7]=(row=3,col=2,val=-6)
a[7]=(4,0,91)	0	j=2 (→3)	b[2]=(row=0,col=4,val=91)
a[8]=(5,2,28)	2	j=5 (→6)	b[5]=(row=2,col=5,val=28)

Final transpose array `b`:

index	row	col	value
b[0]	6	6	8
b[1]	0	0	15
b[2]	0	4	91
b[3]	1	1	11
b[4]	2	1	3
b[5]	2	5	28
b[6]	3	0	22
b[7]	3	2	-6
b[8]	5	0	-15

Sanity check — this exactly matches transposing the matrix element-by-element (e.g., original `a[0][3]=22` correctly becomes transpose `b[3][0]=22`; original `a[4][0]=91` becomes `b[0][4]=91`), and the triples in `b` are already in valid row-major order (rows `0,0,1,2,2,3,3,5` — ascending; columns ascending within each row) — confirming the algorithm produced a correctly ordered result in a single pass, with **no re-sorting needed**.

---

13. Consolidated Exam Question Bank

Definitions to be ready to state precisely: - Algorithm, and its 5 properties. - Data structure; primitive vs non-primitive; linear vs non-linear. - Abstract Data Type (ADT). - Recursion; direct vs indirect recursion; base case. - Pointer; dangling reference; NULL pointer. - `struct` vs `union`. - Self-referential structure. - Sparse matrix.

“Write the ADT specification of…” questions: - Array ADT (§6.1) - Polynomial ADT (§10.2) - Sparse-Matrix ADT (§11.2)

“Write a C program / function for…” questions: - Selection sort (§2.2) and its correctness proof. - Recursive binary search (§3.2). - Recursive permutation generator (§3.3). - `malloc` / `free` usage and avoiding a dangling reference (§5.2–5.3). - Structure equality check function `humans_equal` (§9.2). - `padd` / `attach` for polynomial addition on the shared-array representation (§10.4) — **derive its O(m+n) time complexity**. - `transpose` and `fast_transpose` for a sparse matrix (§11.5) — **derive their time complexities and explain why `fast_transpose` is better**.

“Trace/dry-run this algorithm on the following input” questions: - Trace `fast_transpose` on a given sparse matrix (practice using §12’s method on a *different* matrix as self-study). - Trace `padd` combining two small given polynomials. - Trace the recursive `perm` function on a 3-element list, e.g. `{a, b, c}`.

Comparison / “differentiate” questions: - `struct` vs `union` (§9.3 table). - Representation 1 vs Representation 2 for polynomials (§10.5 table). - 2-D array transpose vs `transpose` vs `fast_transpose`, with time complexities (§11.6 table). - Statically declared arrays vs dynamically allocated arrays (§7). - Row-major vs column-major order (§8.2).

Numerical/derivation questions: - Given `a[i][j]` and a base address, compute the address of a specific element using the row-major addressing formula (§8.2). - Given a sparse matrix, write its array-of-triples representation (§11.4) and then compute its transpose by hand (§12). - Given the sizes of two polynomials (`m`, `n` nonzero terms), state and justify the time complexity of adding them (§10.4).